11. Decorator Design Pattern in Java

11.1 Introduction

The **Decorator Design Pattern** is a **structural pattern** that allows behavior to be added to individual objects dynamically without affecting other objects of the same class.

11.2 Problem it Solves

When subclassing leads to too many subclasses just to support different combinations of behaviour, the decorator pattern offers an elegant alternative.

11.3 What is Decorator Pattern?

Decorator wraps an object and provides additional behavior while conforming to the same interface.

11.4 Real-World Analogy

A coffee shop allows you to add milk, sugar, and whipped cream to a coffee. Each add-on decorates the original coffee without modifying its class.

11.5 Structure

```
package com.mannemlokesh;

//Abstraction
abstract class RemoteControl {
	protected Device device;

	public RemoteControl(Device device) {
		this.device = device;
	}

	public abstract void togglePower();
}

//Implementation Interface
interface Device {
	void turnOn();

	void turnOff();

	boolean isEnabled();
}
```

11.6 Benefits

- Add or remove responsibilities dynamically
- Avoid subclass explosion
- Follows Open/Closed principle

11.7 Implementation Steps

1. Create a component interface

2. Create a concrete component
3. Create an abstract decorator
4. Implement concrete decorators that add behaviour

11.8 Examples

Example1: Coffee with Add-ons

```
package com.mannemlokesh;

interface Coffee {
	String getDescription();

	double getCost();
}

class SimpleCoffee implements Coffee {
	public String getDescription() {
		return "Simple Coffee";
	}

	public double getCost() {
		return 5.0;
	}
}

abstract class CoffeeDecorator implements Coffee {
	protected Coffee coffee;

	public CoffeeDecorator(Coffee coffee) {
		this.coffee = coffee;
	}

	public String getDescription() {
		return coffee.getDescription();
	}

	public double getCost() {
		return coffee.getCost();
	}
}

class MilkDecorator extends CoffeeDecorator {
	public MilkDecorator(Coffee coffee) {
		super(coffee);
	}

	public String getDescription() {
		return super.getDescription() + ", Milk";
	}

	public double getCost() {
		return super.getCost() + 1.5;
	}
}

class SugarDecorator extends CoffeeDecorator {
	public SugarDecorator(Coffee coffee) {
		super(coffee);
```

```
	}

	public String getDescription() {
		return super.getDescription() + ", Sugar";
	}

	public double getCost() {
		return super.getCost() + 0.5;
	}
}

public class Example1CoffeeShop {
	public static void main(String[] args) {
		Coffee coffee = new SimpleCoffee();
		coffee = new MilkDecorator(coffee);
		coffee = new SugarDecorator(coffee);

		System.out.println("Order: " + coffee.getDescription());
		System.out.println("Cost: $" + coffee.getCost());
	}
}
```

Output:

```
Order: Simple Coffee, Milk, Sugar
Cost: $7.0
```

Example2: Text Editor with Formatting

```
package com.mannemlokesh;

interface Text {
	String render();
}

class PlainText implements Text {
	public String render() {
		return "Hello World";
	}
}

abstract class TextDecorator implements Text {
	protected Text text;

	public TextDecorator(Text text) {
		this.text = text;
	}

	public String render() {
		return text.render();
	}
}

class BoldDecorator extends TextDecorator {
	public BoldDecorator(Text text) {
		super(text);
	}
```

```
	public String render() {
		return "<b>" + super.render() + "</b>";
	}
}

class ItalicDecorator extends TextDecorator {
	public ItalicDecorator(Text text) {
		super(text);
	}

	public String render() {
		return "<i>" + super.render() + "</i>";
	}
}

public class Example2TextEditor {
	public static void main(String[] args) {
		Text text = new PlainText();
		text = new BoldDecorator(text);
		text = new ItalicDecorator(text);

		System.out.println(text.render());
	}
}
```

Output:

```
<i><b>Hello World</b></i>
```

Example3: Logging Enhancements

```
package com.mannemlokesh;

interface Logger {
	void log(String message);
}

class ConsoleLogger implements Logger {
	public void log(String message) {
		System.out.println("LOG: " + message);
	}
}

abstract class LoggerDecorator implements Logger {
	protected Logger logger;

	public LoggerDecorator(Logger logger) {
		this.logger = logger;
	}

	public void log(String message) {
		logger.log(message);
	}
}

class TimestampLogger extends LoggerDecorator {
	public TimestampLogger(Logger logger) {
		super(logger);
```

```
        }

        public void log(String message) {
                String timestamp = java.time.LocalDateTime.now().toString();
                super.log(timestamp + " - " + message);
        }
}

class EncryptionLogger extends LoggerDecorator {
        public EncryptionLogger(Logger logger) {
                super(logger);
        }

        public void log(String message) {
                String encrypted = new StringBuilder(message).reverse().toString();
                super.log("Encrypted: " + encrypted);
        }
}

public class Example3LogApp {
        public static void main(String[] args) {
                Logger logger = new ConsoleLogger();
                logger = new TimestampLogger(logger);
                logger = new EncryptionLogger(logger);

                logger.log("User logged in");
        }
}
```

Output:

```
LOG: 2025-06-27T05:55:48.700742800 - Encrypted: ni deggol resU
```

11.9 Decorator vs Other Patterns

Pattern	Purpose
Decorator	Add behaviour to objects dynamically
Adapter	Convert one interface to another
Proxy	Control access or add lazy loading
Composite	Treat group of objects as single object

11.10 Pros and Cons

✅ **Pros**

- Flexible runtime behaviour changes
- Avoids class explosion
- Follows SOLID principles

❌ **Cons**

- Can lead to many small classes
- Debugging may become complex

11.11 When to Use / Not to Use

Use When:

- You need to add behaviour without altering class
- Behaviour needs to be added conditionally or at runtime

Avoid When:

- You can use simple inheritance
- Too many layers of decorators reduce readability

11.12 Summary

Feature	Description
Intent	Add behaviour dynamically
Uses	UI components, stream wrappers, loggers
Key Benefit	Composable behaviour without modifying class

The **Decorator Pattern** is ideal for building extensible and flexible applications where responsibilities need to be added to objects dynamically.